

Source Code - EbookStore (Sistem Penjualan Ebook Digital)

Lampiran Source Code

Daftar File

app/Http/Controllers/AdminController.php
app/Http/Controllers/Auth/AuthenticatedSessionController.php
app/Http/Controllers/Auth/ConfirmablePasswordController.php
app/Http/Controllers/Auth/EmailVerificationNotificationController.php
app/Http/Controllers/Auth/EmailVerificationPromptController.php
app/Http/Controllers/Auth/NewPasswordController.php
app/Http/Controllers/Auth/PasswordController.php
app/Http/Controllers/Auth/PasswordResetLinkController.php
app/Http/Controllers/Auth/RegisteredUserController.php
app/Http/Controllers/Auth/VerifyEmailController.php
app/Http/Controllers/CategoryController.php
app/Http/Controllers/Controller.php
app/Http/Controllers/DownloadController.php
app/Http/Controllers/EbookController.php
app/Http/Controllers/HomeController.php
app/Http/Controllers/OrderController.php
app/Http/Controllers/PembeliController.php
app/Http/Controllers/PenjualController.php
app/Http/Controllers/ProfileController.php
app/Http/Controllers/PromoController.php
app/Http/Controllers/ReviewController.php
app/Http/Controllers/UserController.php
app/Http/Middleware/RoleMiddleware.php
app/Models/Category.php
app/Models/Download.php
app/Models/Ebook.php
app/Models/Order.php
app/Models/Promo.php
app/Models/Review.php
app/Models/User.php
database/migrations/0001_01_01_000000_create_users_table.php
database/migrations/0001_01_01_000001_create_cache_table.php
database/migrations/0001_01_01_000002_create_jobs_table.php
database/migrations/2026_06_10_134147_add_role_to_users_table.php
database/migrations/2026_06_10_134423_create_categories_table.php
database/migrations/2026_06_10_134519_create_ebooks_table.php
database/migrations/2026_06_10_134625_create_orders_table.php
database/migrations/2026_06_10_134701_create_downloads_table.php
database/migrations/2026_06_10_134850_create_reviews_table.php
database/migrations/2026_06_10_134958_create_promos_table.php
database/seeders/CategorySeeder.php
database/seeders/DatabaseSeeder.php
database/seeders/EbookSeeder.php
database/seeders/UserSeeder.php
routes/auth.php
routes/console.php
routes/web.php

app/Http/Controllers/AdminController.php

<?php

```
namespace App\Http\Controllers;

use App\Models\User;
use App\Models\Ebook;
use App\Models\Order;
use App\Models\Category;
use Illuminate\Http\Request;

class AdminController extends Controller
{
    public function dashboard()
    {
        $totalPembeli = User::where('role', 'pembeli')->count();
        $totalPenjual = User::where('role', 'penjual')->count();
        $totalEbook = Ebook::count();
        $totalTransaksi = Order::where('status', 'lunas')->sum('total_price');
        $totalOrder = Order::count();

        $userTerbaru = User::latest()->take(5)->get();
        $ebookTerbaru = Ebook::with(['seller', 'category'])->latest()->take(5)->get();
        $orderTerbaru = Order::with(['buyer', 'ebook'])->latest()->take(5)->get();

        return view('admin.dashboard', compact(
            'totalPembeli', 'totalPenjual', 'totalEbook',
            'totalTransaksi', 'totalOrder',
            'userTerbaru', 'ebookTerbaru', 'orderTerbaru'
        ));
    }

    public function laporan(Request $request)
    {
        $query = Order::where('status', 'lunas')->with(['buyer', 'ebook.seller']);

        if ($request->bulan) {
            $query->whereMonth('created_at', $request->bulan);
        }
        if ($request->tahun) {
            $query->whereYear('created_at', $request->tahun);
        }

        $totalPendapatan = (clone $query)->sum('total_price');
        $orders = $query->latest()->paginate(15);

        $perKategori = Order::where('orders.status', 'lunas')
            ->join('ebooks', 'orders.ebook_id', '=', 'ebooks.id')
            ->join('categories', 'ebooks.category_id', '=', 'categories.id')
            ->selectRaw('categories.name, COUNT(*) as total, SUM(orders.total_price) as pendapatan')
            ->groupBy('categories.name')
            ->get();

        return view('admin.laporan', compact('orders', 'totalPendapatan', 'perKategori'));
    }
}
```

app/Http/Controllers/Auth/AuthenticatedSessionController.php

```
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Http\Requests\Auth\LoginRequest;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\View\View;

class AuthenticatedSessionController extends Controller
{
    /**
     * Display the login view.
     */
    public function create(): View
    {
        return view('auth.login');
    }

    /**
     * Handle an incoming authentication request.
     */
    public function store(LoginRequest $request): RedirectResponse
    {
        $request->authenticate();

        $request->session()->regenerate();

        $role = auth()->user()->role;

        return match($role) {
            'admin' => redirect()->route('admin.dashboard'),
            'penjual' => redirect()->route('penjual.dashboard'),
            default => redirect()->route('pembeli.dashboard'),
        };
    }

    /**
     * Destroy an authenticated session.
     */
    public function destroy(Request $request): RedirectResponse
    {
        Auth::guard('web')->logout();

        $request->session()->invalidate();

        $request->session()->regenerateToken();

        return redirect('/');
    }
}
```

app/Http/Controllers/Auth/ConfirmablePasswordController.php

```
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Validation\ValidationException;
use Illuminate\View\View;

class ConfirmablePasswordController extends Controller
{
    /**
     * Show the confirm password view.
     */
    public function show(): View
    {
        return view('auth.confirm-password');
    }

    /**
     * Confirm the user's password.
     */
    public function store(Request $request): RedirectResponse
    {
        if (! Auth::guard('web')->validate([
            'email' => $request->user()->email,
            'password' => $request->password,
        ])) {
            throw ValidationException::withMessages([
                'password' => __('auth.password'),
            ]);
        }

        $request->session()->put('auth.password_confirmed_at', time());

        return redirect()->intended(route('dashboard', absolute: false));
    }
}
```

app/Http/Controllers/Auth/EmailVerificationNotificationController.php

```
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;

class EmailVerificationNotificationController extends Controller
{
    /**
     * Send a new email verification notification.
     */
    public function store(Request $request): RedirectResponse
    {
        if ($request->user()->hasVerifiedEmail()) {
            return redirect()->intended(route('dashboard', absolute: false));
        }

        $request->user()->sendEmailVerificationNotification();

        return back()->with('status', 'verification-link-sent');
    }
}
```

app/Http/Controllers/Auth/EmailVerificationPromptController.php

```
<?php
```

```
namespace App\Http\Controllers\Auth;
```

```
use App\Http\Controllers\Controller;  
use Illuminate\Http\RedirectResponse;  
use Illuminate\Http\Request;  
use Illuminate\View\View;
```

```
class EmailVerificationPromptController extends Controller
```

```
{  
    /**  
     * Display the email verification prompt.  
     */  
    public function __invoke(Request $request): RedirectResponse|View  
    {  
        return $request->user()->hasVerifiedEmail()  
            ? redirect()->intended(route('dashboard', absolute: false))  
            : view('auth.verify-email');  
    }  
}
```

app/Http/Controllers/Auth/NewPasswordController.php

<?php

```
namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Models\User;
use Illuminate\Auth\Events\PasswordReset;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Hash;
use Illuminate\Support\Facades\Password;
use Illuminate\Support\Str;
use Illuminate\Validation\Rules;
use Illuminate\Validation\ValidationException;
use Illuminate\View\View;

class NewPasswordController extends Controller
{
    /**
     * Display the password reset view.
     */
    public function create(Request $request): View
    {
        return view('auth.reset-password', ['request' => $request]);
    }

    /**
     * Handle an incoming new password request.
     *
     * @throws ValidationException
     */
    public function store(Request $request): RedirectResponse
    {
        $request->validate([
            'token' => ['required'],
            'email' => ['required', 'email'],
            'password' => ['required', 'confirmed', Rules\Password::defaults()],
        ]);

        // Here we will attempt to reset the user's password. If it is successful we
        // will update the password on an actual user model and persist it to the
        // database. Otherwise we will parse the error and return the response.
        $status = Password::reset(
            $request->only('email', 'password', 'password_confirmation', 'token'),
            function (User $user) use ($request) {
                $user->forceFill([
                    'password' => Hash::make($request->password),
                    'remember_token' => Str::random(60),
                ])->save();

                event(new PasswordReset($user));
            }
        );

        // If the password was successfully reset, we will redirect the user back to
        // the application's home authenticated view. If there is an error we can
        // redirect them back to where they came from with their error message.
        return $status == Password::PASSWORD_RESET
            ? redirect()->route('login')->with('status', __($status))
            : back()->withInput($request->only('email'))
                ->withErrors(['email' => __($status)]);
    }
}
```


app/Http/Controllers/Auth/PasswordController.php

```
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Hash;
use Illuminate\Validation\Rules\Password;

class PasswordController extends Controller
{
    /**
     * Update the user's password.
     */
    public function update(Request $request): RedirectResponse
    {
        $validated = $request->validateWithBag('updatePassword', [
            'current_password' => ['required', 'current_password'],
            'password' => ['required', Password::defaults(), 'confirmed'],
        ]);

        $request->user()->update([
            'password' => Hash::make($validated['password']),
        ]);

        return back()->with('status', 'password-updated');
    }
}
```

app/Http/Controllers/Auth/PasswordResetLinkController.php

```
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Password;
use Illuminate\Validation\ValidationException;
use Illuminate\View\View;

class PasswordResetLinkController extends Controller
{
    /**
     * Display the password reset link request view.
     */
    public function create(): View
    {
        return view('auth.forgot-password');
    }

    /**
     * Handle an incoming password reset link request.
     *
     * @throws ValidationException
     */
    public function store(Request $request): RedirectResponse
    {
        $request->validate([
            'email' => ['required', 'email'],
        ]);

        // We will send the password reset link to this user. Once we have attempted
        // to send the link, we will examine the response then see the message we
        // need to show to the user. Finally, we'll send out a proper response.
        $status = Password::sendResetLink(
            $request->only('email')
        );

        return $status == Password::RESET_LINK_SENT
            ? back()->with('status', __($status))
            : back()->withInput($request->only('email'))
                ->withErrors(['email' => __($status)]);
    }
}
```

app/Http/Controllers/Auth/RegisteredUserController.php

```
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Models\User;
use Illuminate\Auth\Events\Registered;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Hash;
use Illuminate\Validation\Rules;
use Illuminate\View\View;

class RegisteredUserController extends Controller
{
    /**
     * Display the registration view.
     */
    public function create(): View
    {
        return view('auth.register');
    }

    /**
     * Handle an incoming registration request.
     */
    public function store(Request $request): RedirectResponse
    {
        $request->validate([
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'string', 'lowercase', 'email', 'max:255', 'unique:'.User::class],
            'password' => ['required', 'confirmed', Rules\Password::defaults()],
        ]);

        $user = User::create([
            'name' => $request->name,
            'email' => $request->email,
            'password' => Hash::make($request->password),
            'role' => 'pembeli', // ← selalu pembeli
        ]);

        event(new Registered($user));

        Auth::login($user);

        // Langsung redirect ke dashboard pembeli
        return redirect()->route('pembeli.dashboard');
    }
}
```

app/Http/Controllers/Auth/VerifyEmailController.php

```
<?php
```

```
namespace App\Http\Controllers\Auth;
```

```
use App\Http\Controllers\Controller;
```

```
use Illuminate\Auth\Events\Verified;
```

```
use Illuminate\Foundation\Auth\EmailVerificationRequest;
```

```
use Illuminate\Http\RedirectResponse;
```

```
class VerifyEmailController extends Controller
```

```
{
```

```
    /**
```

```
     * Mark the authenticated user's email address as verified.
```

```
    */
```

```
    public function __invoke(EmailVerificationRequest $request): RedirectResponse
```

```
    {
```

```
        if ($request->user()->hasVerifiedEmail()) {
```

```
            return redirect()->intended(route('dashboard', absolute: false).'?verified=1');
```

```
        }
```

```
        if ($request->user()->markEmailAsVerified()) {
```

```
            event(new Verified($request->user()));
```

```
        }
```

```
        return redirect()->intended(route('dashboard', absolute: false).'?verified=1');
```

```
    }
```

```
}
```

app/Http/Controllers/CategoryController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\Category;
use Illuminate\Http\Request;
use Illuminate\Support\Str;

class CategoryController extends Controller
{
    public function index()
    {
        $categories = Category::withCount('ebooks')->latest()->paginate(15);
        return view('admin.kategori.index', compact('categories'));
    }

    public function create()
    {
        return view('admin.kategori.create');
    }

    public function store(Request $request)
    {
        $request->validate([
            'name' => 'required|string|max:255|unique:categories,name',
        ]);

        Category::create([
            'name' => $request->name,
            'slug' => Str::slug($request->name),
        ]);

        return redirect()->route('admin.kategori.index')
            ->with('success', 'Kategori berhasil ditambahkan!');
    }

    public function edit(Category $kategori)
    {
        return view('admin.kategori.edit', compact('kategori'));
    }

    public function update(Request $request, Category $kategori)
    {
        $request->validate([
            'name' => 'required|string|max:255|unique:categories,name,'.$kategori->id,
        ]);

        $kategori->update([
            'name' => $request->name,
            'slug' => Str::slug($request->name),
        ]);

        return redirect()->route('admin.kategori.index')
            ->with('success', 'Kategori berhasil diperbarui!');
    }

    public function destroy(Category $kategori)
    {
        $kategori->delete();
        return back()->with('success', 'Kategori berhasil dihapus!');
    }
}
```

app/Http/Controllers/Controller.php

```
<?php

namespace App\Http\Controllers;

abstract class Controller
{
    //
}
```

app/Http/Controllers/DownloadController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\Order;
use App\Models\Download;
use Illuminate\Support\Facades\Storage;

class DownloadController extends Controller
{
    public function download(Order $order)
    {
        // Pastikan order milik user ini
        abort_if($order->buyer_id !== auth()->id(), 403);

        // Pastikan sudah lunas
        abort_if($order->status !== 'lunas', 403, 'Pembayaran belum dikonfirmasi.');
```

// Catat log download
Download::create([
 'order_id' => \$order->id,
 'buyer_id' => auth()->id(),
 'ebook_id' => \$order->ebook_id,
 'downloaded_at' => now(),
]);

// Download file dari private storage
\$filePath = \$order->ebook->file;

abort_if(!Storage::disk('private')->exists(\$filePath), 404, 'File tidak ditemukan.');

return Storage::disk('private')->download(
 \$filePath,
 \$order->ebook->title . '.pdf'
);

}

app/Http/Controllers/EbookController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\Ebook;
use App\Models\Order;
use App\Models\Category;
use Illuminate\Http\Request;
use Illuminate\Support\Str;
use Illuminate\Support\Facades\Storage;

class EbookController extends Controller
{
    public function index()
    {
        $ebooks = Ebook::where('seller_id', auth()->id())
            ->with('category')
            ->latest()
            ->paginate(10);

        return view('penjual.ebook.index', compact('ebooks'));
    }

    public function create()
    {
        $categories = Category::all();
        return view('penjual.ebook.create', compact('categories'));
    }

    public function store(Request $request)
    {
        $request->validate([
            'title' => 'required|string|max:255',
            'category_id' => 'required|exists:categories,id',
            'description' => 'required|string',
            'price' => 'required|numeric|min:0',
            'cover' => 'nullable|image|mimes:jpg,jpeg,png|max:2048',
            'file' => 'required|mimes:pdf|max:51200',
            'status' => 'required|in:aktif,nonaktif',
        ]);

        $coverPath = null;
        if ($request->hasFile('cover')) {
            $coverPath = $request->file('cover')->store('covers', 'public');
        }

        $filePath = $request->file('file')->store('ebooks', 'private');

        Ebook::create([
            'seller_id' => auth()->id(),
            'category_id' => $request->category_id,
            'title' => $request->title,
            'slug' => Str::slug($request->title) . '-' . Str::random(5),
            'description' => $request->description,
            'price' => $request->price,
            'cover' => $coverPath,
            'file' => $filePath,
            'status' => $request->status,
        ]);

        return redirect()->route('penjual.ebook.index')
            ->with('success', 'Ebook berhasil ditambahkan!');
    }

    public function edit(Ebook $ebook)
    {
        abort_if($ebook->seller_id !== auth()->id(), 403);
        $categories = Category::all();
        return view('penjual.ebook.edit', compact('ebook', 'categories'));
    }

    public function update(Request $request, Ebook $ebook)
    {
        abort_if($ebook->seller_id !== auth()->id(), 403);

        $request->validate([
            'title' => 'required|string|max:255',
            'category_id' => 'required|exists:categories,id',
            'description' => 'required|string',
            'price' => 'required|numeric|min:0',
```



```

        'cover'      => 'nullable|image|mimes:jpg,jpeg,png|max:2048',
        'file'       => 'nullable|mimes:pdf|max:51200',
        'status'     => 'required|in:aktif,nonaktif',
    ]);

    if ($request->hasFile('cover')) {
        if ($ebook->cover) Storage::disk('public')->delete($ebook->cover);
        $ebook->cover = $request->file('cover')->store('covers', 'public');
    }

    if ($request->hasFile('file')) {
        Storage::disk('private')->delete($ebook->file);
        $ebook->file = $request->file('file')->store('ebooks', 'private');
    }

    $ebook->update([
        'category_id' => $request->category_id,
        'title'       => $request->title,
        'slug'        => Str::slug($request->title) . '-' . Str::random(5),
        'description' => $request->description,
        'price'       => $request->price,
        'cover'       => $ebook->cover,
        'file'        => $ebook->file,
        'status'      => $request->status,
    ]);

    return redirect()->route('penjual.ebook.index')
        ->with('success', 'Ebook berhasil diperbarui!');
}

public function destroy(Ebook $ebook)
{
    abort_if($ebook->seller_id !== auth()->id(), 403);

    if ($ebook->cover) Storage::disk('public')->delete($ebook->cover);
    Storage::disk('private')->delete($ebook->file);
    $ebook->delete();

    return redirect()->route('penjual.ebook.index')
        ->with('success', 'Ebook berhasil dihapus!');
}

public function show(string $slug)
{
    $ebook = Ebook::where('slug', $slug)
        ->where('status', 'aktif')
        ->with(['category', 'seller', 'reviews.buyer', 'promo'])
        ->firstOrFail();

    $sudahBeli = false;
    if (auth()->check()) {
        $sudahBeli = Order::where('buyer_id', auth()->id())
            ->where('ebook_id', $ebook->id)
            ->where('status', 'lunas')
            ->exists();
    }

    return view('ebook.show', compact('ebook', 'sudahBeli'));
}
}

```

app/Http/Controllers/HomeController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\Ebook;
use App\Models\Category;
use Illuminate\Http\Request;

class HomeController extends Controller
{
    public function index(Request $request)
    {
        $query = Ebook::where('status', 'aktif')
            ->with(['category', 'seller', 'promo']);

        if ($request->search) {
            $query->where(function($q) use ($request) {
                $q->where('title', 'like', '%'.$request->search.'%')
                    ->orWhere('description', 'like', '%'.$request->search.'%')
                    ->orWhereHas('seller', fn($q) =>
                        $q->where('name', 'like', '%'.$request->search.'%')
                    );
            });
        }

        if ($request->category) {
            $query->whereHas('category', fn($q) =>
                $q->where('slug', $request->category)
            );
        }

        $ebooks = $query->latest()->paginate(8)->withQueryString();
        $categories = Category::withCount('ebooks')->get();

        return view('home.index', compact('ebooks', 'categories'));
    }

    public function kategori(string $slug)
    {
        $category = Category::where('slug', $slug)->firstOrFail();

        $ebooks = Ebook::where('status', 'aktif')
            ->where('category_id', $category->id)
            ->with(['category', 'seller', 'promo'])
            ->latest()
            ->paginate(12);

        return view('home.kategori', compact('ebooks', 'category'));
    }
}
```

app/Http/Controllers/OrderController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\Order;
use App\Models\Ebook;
use Illuminate\Http\Request;

class OrderController extends Controller
{
    // Riwayat pembelian pembeli
    public function index()
    {
        $orders = Order::where('buyer_id', auth()->id())
            ->with(['ebook.category'])
            ->latest()
            ->paginate(10);

        return view('pembeli.riwayat', compact('orders'));
    }

    // Proses beli ebook
    public function store(Request $request, Ebook $ebook)
    {
        // Cek ebook aktif
        abort_if($ebook->status !== 'aktif', 404);

        // Cek sudah pernah beli
        $sudahBeli = Order::where('buyer_id', auth()->id())
            ->where('ebook_id', $ebook->id)
            ->whereIn('status', ['pending', 'lunas'])
            ->exists();

        if ($sudahBeli) {
            return back()->with('error', 'Kamu sudah membeli atau sedang memproses ebook ini.');
```

```
        }

        // Hitung harga final
        $harga = $ebook->promo ? $ebook->final_price : $ebook->price;

        Order::create([
            'buyer_id'     => auth()->id(),
            'ebook_id'     => $ebook->id,
            'total_price'  => $harga,
            'status'       => 'pending',
        ]);

        return redirect()->route('pembeli.order.index')
            ->with('success', 'Pesanan berhasil dibuat! Silakan upload bukti pembayaran.');
```

```
    }

    // Upload bukti pembayaran
    public function uploadBukti(Request $request, Order $order)
    {
        abort_if($order->buyer_id !== auth()->id(), 403);
        abort_if($order->status !== 'pending', 403);

        $request->validate([
            'payment_proof' => 'required|image|mimes:jpg,jpeg,png|max:2048',
        ]);

        $path = $request->file('payment_proof')->store('payment_proofs', 'public');

        $order->update(['payment_proof' => $path]);

        return back()->with('success', 'Bukti pembayaran berhasil diupload! Menunggu konfirmasi penjual.');
```

```
    }
}
```

app/Http/Controllers/PembeliController.php

```
<?php
```

```
namespace App\Http\Controllers;
```

```
use App\Models\Order;
```

```
use App\Models\Ebook;
```

```
use Illuminate\Http\Request;
```

```
class PembeliController extends Controller
```

```
{
```

```
    public function dashboard()
```

```
    {
```

```
        $totalBeli    = Order::where('buyer_id', auth()->id()->count();
```

```
        $totalLunas   = Order::where('buyer_id', auth()->id()->where('status', 'lunas')->count();
```

```
        $totalPending = Order::where('buyer_id', auth()->id()->where('status', 'pending')->count();
```

```
        $orderTerbaru = Order::where('buyer_id', auth()->id())
```

```
            ->with(['ebook.category'])
```

```
            ->latest()->take(5)->get();
```

```
        $rekomendasiEbook = Ebook::where('status', 'aktif')
```

```
            ->whereNotIn('id', Order::where('buyer_id', auth()->id())
```

```
                ->where('status', 'lunas')->pluck('ebook_id'))
```

```
            ->with(['category', 'promo'])
```

```
            ->latest()->take(4)->get();
```

```
        return view('pembeli.dashboard', compact(
```

```
            'totalBeli', 'totalLunas', 'totalPending',
```

```
            'orderTerbaru', 'rekomendasiEbook'
```

```
        ));
```

```
    }
```

```
}
```

app/Http/Controllers/PenjualController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\Ebook;
use App\Models\Order;
use Illuminate\Http\Request;

class PenjualController extends Controller
{
    public function dashboard()
    {
        $totalEbook = Ebook::where('seller_id', auth()->id()->count();

        $totalPenjualan = Order::whereHas('ebook', fn($q) =>
            $q->where('seller_id', auth()->id())
            ->where('status', 'lunas')->count();

        $totalPending = Order::whereHas('ebook', fn($q) =>
            $q->where('seller_id', auth()->id())
            ->where('status', 'pending')->count();

        $totalPendapatan = Order::whereHas('ebook', fn($q) =>
            $q->where('seller_id', auth()->id())
            ->where('status', 'lunas')->sum('total_price');

        $orderTerbaru = Order::whereHas('ebook', fn($q) =>
            $q->where('seller_id', auth()->id())
            ->with(['ebook', 'buyer'])
            ->latest()->take(5)->get();

        return view('penjual.dashboard', compact(
            'totalEbook', 'totalPenjualan', 'totalPending',
            'totalPendapatan', 'orderTerbaru'
        ));
    }

    public function penjualan()
    {
        $orders = Order::whereHas('ebook', fn($q) =>
            $q->where('seller_id', auth()->id())
            ->with(['ebook', 'buyer'])
            ->latest()->paginate(15);

        return view('penjual.penjualan', compact('orders'));
    }

    public function updateStatus(Request $request, Order $order)
    {
        abort_if($order->ebook->seller_id !== auth()->id(), 403);

        $request->validate([
            'status' => 'required|in:lunas,ditolak'
        ]);

        $order->update(['status' => $request->status]);

        return back()->with('success', 'Status order berhasil diperbarui!');
    }

    public function laporan(Request $request)
    {
        $query = Order::whereHas('ebook', fn($q) =>
            $q->where('seller_id', auth()->id())
            ->where('status', 'lunas')
            ->with(['ebook', 'buyer']));

        if ($request->bulan) {
            $query->whereMonth('created_at', $request->bulan);
        }
        if ($request->tahun) {
            $query->whereYear('created_at', $request->tahun);
        }

        $totalPendapatan = (clone $query)->sum('total_price');
        $orders = $query->latest()->paginate(15);

        return view('penjual.laporan', compact('orders', 'totalPendapatan'));
    }
}
```

app/Http/Controllers/ProfileController.php

```
<?php

namespace App\Http\Controllers;

use App\Http\Requests\ProfileUpdateRequest;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Redirect;
use Illuminate\View\View;

class ProfileController extends Controller
{
    /**
     * Display the user's profile form.
     */
    public function edit(Request $request): View
    {
        return view('profile.edit', [
            'user' => $request->user(),
        ]);
    }

    /**
     * Update the user's profile information.
     */
    public function update(ProfileUpdateRequest $request): RedirectResponse
    {
        $request->user()->fill($request->validated());

        if ($request->user()->isDirty('email')) {
            $request->user()->email_verified_at = null;
        }

        $request->user()->save();

        return Redirect::route('profile.edit')->with('status', 'profile-updated');
    }

    /**
     * Delete the user's account.
     */
    public function destroy(Request $request): RedirectResponse
    {
        $request->validateWithBag('userDeletion', [
            'password' => ['required', 'current_password'],
        ]);

        $user = $request->user();

        Auth::logout();

        $user->delete();

        $request->session()->invalidate();
        $request->session()->regenerateToken();

        return Redirect::to('/');
    }
}
```

app/Http/Controllers/PromoController.php

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class PromoController extends Controller
{
    /**
     * Display a listing of the resource.
     */
    public function index()
    {
        //
    }

    /**
     * Show the form for creating a new resource.
     */
    public function create()
    {
        //
    }

    /**
     * Store a newly created resource in storage.
     */
    public function store(Request $request)
    {
        //
    }

    /**
     * Display the specified resource.
     */
    public function show(string $id)
    {
        //
    }

    /**
     * Show the form for editing the specified resource.
     */
    public function edit(string $id)
    {
        //
    }

    /**
     * Update the specified resource in storage.
     */
    public function update(Request $request, string $id)
    {
        //
    }

    /**
     * Remove the specified resource from storage.
     */
    public function destroy(string $id)
    {
        //
    }
}
```

app/Http/Controllers/ReviewController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\Review;
use App\Models\Order;
use App\Models\Ebook;
use Illuminate\Http\Request;

class ReviewController extends Controller
{
    public function store(Request $request, Ebook $ebook)
    {
        // Pastikan sudah beli dan lunas
        $sudahBeli = Order::where('buyer_id', auth()->id())
            ->where('ebook_id', $ebook->id)
            ->where('status', 'lunas')
            ->exists();

        if (!$sudahBeli) {
            return back()->with('error', 'Kamu harus membeli ebook ini sebelum memberi ulasan.');
```

```
        }

        // Cek sudah pernah review
        $sudahReview = Review::where('buyer_id', auth()->id())
            ->where('ebook_id', $ebook->id)
            ->exists();

        if ($sudahReview) {
            return back()->with('error', 'Kamu sudah memberikan ulasan untuk ebook ini.');
```

```
        }

        $request->validate([
            'rating' => 'required|integer|min:1|max:5',
            'comment' => 'nullable|string|max:500',
        ]);

        Review::create([
            'ebook_id' => $ebook->id,
            'buyer_id' => auth()->id(),
            'rating' => $request->rating,
            'comment' => $request->comment,
        ]);

        return back()->with('success', 'Ulasan berhasil dikirim!');
    }
}
```


app/Http/Controllers/UserController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\User;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Hash;

class UserController extends Controller
{
    public function index(Request $request)
    {
        $query = User::query();

        if ($request->role) {
            $query->where('role', $request->role);
        }
        if ($request->search) {
            $query->where(function($q) use ($request) {
                $q->where('name', 'like', '%'.$request->search.'%')
                    ->orWhere('email', 'like', '%'.$request->search.'%');
            });
        }

        $users = $query->latest()->paginate(15);
        return view('admin.user.index', compact('users'));
    }

    public function create()
    {
        return view('admin.user.create');
    }

    public function store(Request $request)
    {
        $request->validate([
            'name' => 'required|string|max:255',
            'email' => 'required|email|unique:users,email',
            'password' => 'required|min:6|confirmed',
            'role' => 'required|in:pembeli,penjual,admin',
        ]);

        User::create([
            'name' => $request->name,
            'email' => $request->email,
            'password' => Hash::make($request->password),
            'role' => $request->role,
        ]);

        return redirect()->route('admin.user.index')
            ->with('success', 'User berhasil ditambahkan!');
    }

    public function edit(User $user)
    {
        return view('admin.user.edit', compact('user'));
    }

    public function update(Request $request, User $user)
    {
        $request->validate([
            'name' => 'required|string|max:255',
            'email' => 'required|email|unique:users,email, '.$user->id,
            'role' => 'required|in:pembeli,penjual,admin',
        ]);

        $user->update([
            'name' => $request->name,
            'email' => $request->email,
            'role' => $request->role,
        ]);

        if ($request->filled('password')) {
            $request->validate(['password' => 'min:6|confirmed']);
            $user->update(['password' => Hash::make($request->password)]);
        }

        return redirect()->route('admin.user.index')
            ->with('success', 'User berhasil diperbarui!');
    }
}
```

```
public function destroy(User $user)
{
    abort_if($user->id === auth()->id(), 403, 'Tidak bisa hapus akun sendiri.');
```

`$user->delete();`

```
    return back()->with('success', 'User berhasil dihapus!');
}
```

app/Http/Middleware/RoleMiddleware.php

```
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;

class RoleMiddleware
{
    public function handle(Request $request, Closure $next, string ...$roles): mixed
    {
        if (!auth()->check() || !in_array(auth()->user()->role, $roles)) {
            abort(403, 'Akses ditolak.');
```

app/Models/Category.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Category extends Model
{
    protected $fillable = ['name', 'slug'];

    public function ebooks()
    {
        return $this->hasMany(Ebook::class);
    }
}
```

app/Models/Download.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Download extends Model
{
    protected $fillable = [
        'order_id', 'buyer_id', 'ebook_id', 'downloaded_at'
    ];

    public function order()
    {
        return $this->belongsTo(Order::class);
    }

    public function buyer()
    {
        return $this->belongsTo(User::class, 'buyer_id');
    }

    public function ebook()
    {
        return $this->belongsTo(Ebook::class);
    }
}
```

app/Models/Ebook.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Ebook extends Model
{
    protected $fillable = [
        'seller_id', 'category_id', 'title', 'slug',
        'description', 'price', 'cover', 'file', 'status'
    ];

    public function seller()
    {
        return $this->belongsTo(User::class, 'seller_id');
    }

    public function category()
    {
        return $this->belongsTo(Category::class);
    }

    public function orders()
    {
        return $this->hasMany(Order::class);
    }

    public function reviews()
    {
        return $this->hasMany(Review::class);
    }

    public function promo()
    {
        return $this->hasOne(Promo::class)
            ->whereDate('start_date', '<=', now())
            ->whereDate('end_date', '>=', now());
    }

    // Harga setelah diskon
    public function getFinalPriceAttribute(): float
    {
        if ($this->promo) {
            return $this->price - ($this->price * $this->promo->discount_percent / 100);
        }
        return (float) $this->price;
    }
}
```

app/Models/Order.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Order extends Model
{
    protected $fillable = [
        'buyer_id', 'ebook_id', 'total_price', 'status', 'payment_proof'
    ];

    public function buyer()
    {
        return $this->belongsTo(User::class, 'buyer_id');
    }

    public function ebook()
    {
        return $this->belongsTo(Ebook::class);
    }

    public function download()
    {
        return $this->hasOne(Download::class);
    }

    public function isLunas(): bool
    {
        return $this->status === 'lunas';
    }
}
```

app/Models/Promo.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Promo extends Model
{
    protected $fillable = [
        'ebook_id', 'discount_percent', 'start_date', 'end_date'
    ];

    protected $casts = [
        'start_date' => 'date',
        'end_date'   => 'date',
    ];

    public function ebook()
    {
        return $this->belongsTo(Ebook::class);
    }
}
```


app/Models/Review.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Review extends Model
{
    protected $fillable = [
        'ebook_id', 'buyer_id', 'rating', 'comment'
    ];

    public function ebook()
    {
        return $this->belongsTo(Ebook::class);
    }

    public function buyer()
    {
        return $this->belongsTo(User::class, 'buyer_id');
    }
}
```

app/Models/User.php

```
<?php

namespace App\Models;

// use Illuminate\Contracts\Auth\MustVerifyEmail;
use Database\Factories\UserFactory;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

class User extends Authenticatable
{
    /** @use HasFactory<UserFactory> */
    use HasFactory, Notifiable;

    /**
     * The attributes that are mass assignable.
     *
     * @var list<string>
     */
    // app/Models/User.php
    protected $fillable = ['name', 'email', 'password', 'role'];

    // Relasi
    public function ebooks()
    {
        return $this->hasMany(Ebook::class, 'seller_id');
    }

    public function orders()
    {
        return $this->hasMany(Order::class, 'buyer_id');
    }

    public function reviews()
    {
        return $this->hasMany(Review::class, 'buyer_id');
    }

    // Helper cek role
    public function isAdmin(): bool { return $this->role === 'admin'; }
    public function isPenjual(): bool { return $this->role === 'penjual'; }
    public function isPembeli(): bool { return $this->role === 'pembeli'; }
}
```

database/migrations/0001_01_01_000000_create_users_table.php

<?php

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('users', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('email')->unique();
            $table->timestamp('email_verified_at')->nullable();
            $table->string('password');
            $table->rememberToken();
            $table->timestamps();
        });

        Schema::create('password_reset_tokens', function (Blueprint $table) {
            $table->string('email')->primary();
            $table->string('token');
            $table->timestamp('created_at')->nullable();
        });

        Schema::create('sessions', function (Blueprint $table) {
            $table->string('id')->primary();
            $table->foreignId('user_id')->nullable()->index();
            $table->string('ip_address', 45)->nullable();
            $table->text('user_agent')->nullable();
            $table->longText('payload');
            $table->integer('last_activity')->index();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('users');
        Schema::dropIfExists('password_reset_tokens');
        Schema::dropIfExists('sessions');
    }
};
```

database/migrations/0001_01_01_000001_create_cache_table.php

<?php

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('cache', function (Blueprint $table) {
            $table->string('key')->primary();
            $table->mediumText('value');
            $table->integer('expiration')->index();
        });

        Schema::create('cache_locks', function (Blueprint $table) {
            $table->string('key')->primary();
            $table->string('owner');
            $table->integer('expiration')->index();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('cache');
        Schema::dropIfExists('cache_locks');
    }
};
```

database/migrations/0001_01_01_000002_create_jobs_table.php

<?php

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('jobs', function (Blueprint $table) {
            $table->id();
            $table->string('queue')->index();
            $table->longText('payload');
            $table->unsignedTinyInteger('attempts');
            $table->unsignedInteger('reserved_at')->nullable();
            $table->unsignedInteger('available_at');
            $table->unsignedInteger('created_at');
        });

        Schema::create('job_batches', function (Blueprint $table) {
            $table->string('id')->primary();
            $table->string('name');
            $table->integer('total_jobs');
            $table->integer('pending_jobs');
            $table->integer('failed_jobs');
            $table->longText('failed_job_ids');
            $table->mediumText('options')->nullable();
            $table->integer('cancelled_at')->nullable();
            $table->integer('created_at');
            $table->integer('finished_at')->nullable();
        });

        Schema::create('failed_jobs', function (Blueprint $table) {
            $table->id();
            $table->string('uuid')->unique();
            $table->text('connection');
            $table->text('queue');
            $table->longText('payload');
            $table->longText('exception');
            $table->timestamp('failed_at')->useCurrent();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('jobs');
        Schema::dropIfExists('job_batches');
        Schema::dropIfExists('failed_jobs');
    }
};
```

database/migrations/2026_06_10_134147_add_role_to_users_table.php

<?php

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    // database/migrations/xxxx_add_role_to_users_table.php
    public function up(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->enum('role', ['pembeli', 'penjual', 'admin'])->default('pembeli')->after('email');
        });
    }

    public function down(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->dropColumn('role');
        });
    }
};
```

database/migrations/2026_06_10_134423_create_categories_table.php

<?php

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('categories', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('slug')->unique();
            $table->timestamps();
        });
    }
};
```

database/migrations/2026_06_10_134519_create_ebooks_table.php

<?php

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('ebooks', function (Blueprint $table) {
            $table->id();
            $table->foreignId('seller_id')->constrained('users')->onDelete('cascade');
            $table->foreignId('category_id')->constrained()->onDelete('cascade');
            $table->string('title');
            $table->string('slug')->unique();
            $table->text('description');
            $table->decimal('price', 10, 2);
            $table->string('cover')->nullable();
            $table->string('file');
            $table->enum('status', ['aktif', 'nonaktif'])->default('aktif');
            $table->timestamps();
        });
    }
};
```


database/migrations/2026_06_10_134625_create_orders_table.php

<?php

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('orders', function (Blueprint $table) {
            $table->id();
            $table->foreignId('buyer_id')->constrained('users')->onDelete('cascade');
            $table->foreignId('ebook_id')->constrained()->onDelete('cascade');
            $table->decimal('total_price', 10, 2);
            $table->enum('status', ['pending', 'lunas', 'ditolak'])->default('pending');
            $table->string('payment_proof')->nullable();
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('orders');
    }
};
```

database/migrations/2026_06_10_134701_create_downloads_table.php

<?php

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('downloads', function (Blueprint $table) {
            $table->id();
            $table->foreignId('order_id')->constrained()->onDelete('cascade');
            $table->foreignId('buyer_id')->constrained('users')->onDelete('cascade');
            $table->foreignId('ebook_id')->constrained()->onDelete('cascade');
            $table->timestamp('downloaded_at')->useCurrent();
            $table->timestamps();
        });
    }
};
```

database/migrations/2026_06_10_134850_create_reviews_table.php

<?php

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('reviews', function (Blueprint $table) {
            $table->id();
            $table->foreignId('ebook_id')->constrained()->onDelete('cascade');
            $table->foreignId('buyer_id')->constrained('users')->onDelete('cascade');
            $table->tinyInteger('rating'); // 1-5
            $table->text('comment')->nullable();
            $table->timestamps();
        });
    }
};
```

database/migrations/2026_06_10_134958_create_promos_table.php

<?php

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('promos', function (Blueprint $table) {
            $table->id();
            $table->foreignId('ebook_id')->constrained()->onDelete('cascade');
            $table->unsignedTinyInteger('discount_percent'); // 1-100
            $table->date('start_date');
            $table->date('end_date');
            $table->timestamps();
        });
    }
};
```

database/seeder/CategorySeeder.php

```
<?php

namespace Database\Seeders;

use Illuminate\Database\Seeder;
use App\Models\Category;

class CategorySeeder extends Seeder
{
    public function run(): void
    {
        $categories = [
            ['name' => 'Teknologi',      'slug' => 'teknologi'],
            ['name' => 'Bisnis',          'slug' => 'bisnis'],
            ['name' => 'Desain',          'slug' => 'desain'],
            ['name' => 'Produktivitas',   'slug' => 'produktivitas'],
            ['name' => 'Pendidikan',      'slug' => 'pendidikan'],
            ['name' => 'Pemrograman',     'slug' => 'pemrograman'],
        ];

        foreach ($categories as $cat) {
            Category::create($cat);
        }
    }
}
```

database/seeder/DatabaseSeeder.php

```
<?php

namespace Database\Seeders;

use Illuminate\Database\Seeder;

class DatabaseSeeder extends Seeder
{
    public function run(): void
    {
        $this->call([
            UserSeeder::class,
            CategorySeeder::class,
            EbookSeeder::class,
        ]);
    }
}
```

database/seeder/EbookSeeder.php

```
<?php
```

```
namespace Database\Seeders;
```

```
use Illuminate\Database\Seeder;
use App\Models\Ebook;
use App\Models\User;
use App\Models\Category;
use Illuminate\Support\Str;
```

```
class EbookSeeder extends Seeder
```

```
{
    public function run(): void
    {
        $penjual = User::where('role', 'penjual')->first();
        $teknologi = Category::where('slug', 'teknologi')->first();
        $bisnis = Category::where('slug', 'bisnis')->first();
        $desain = Category::where('slug', 'desain')->first();
        $pemrograman = Category::where('slug', 'pemrograman')->first();

        $ebooks = [
            [
                'title' => 'Machine Learning dengan Python',
                'description' => 'Panduan lengkap belajar Machine Learning menggunakan Python. Mulai dari regresi linier,
                'price' => 75000,
                'category_id' => $teknologi->id,
            ],
            [
                'title' => 'Panduan Desain UI/UX Modern',
                'description' => 'Pelajari prinsip desain UI/UX modern yang digunakan oleh designer profesional. Dari wire
                'price' => 55000,
                'category_id' => $desain->id,
            ],
            [
                'title' => 'Sukses Bisnis Online dari Nol',
                'description' => 'Strategi membangun bisnis online yang profitable dari nol. Meliputi riset pasar, brandin
                'price' => 49000,
                'category_id' => $bisnis->id,
            ],
            [
                'title' => 'Laravel untuk Web Developer',
                'description' => 'Kuasai Laravel framework PHP terpopuler. Dari routing, MVC, Eloquent ORM, hingga deploy
                'price' => 85000,
                'category_id' => $pemrograman->id,
            ],
            [
                'title' => 'Flutter dari Nol sampai Mahir',
                'description' => 'Belajar membuat aplikasi mobile cross-platform dengan Flutter. Dilengkapi dengan contoh
                'price' => 65000,
                'category_id' => $teknologi->id,
            ],
            [
                'title' => 'Digital Marketing Strategy 2026',
                'description' => 'Strategi digital marketing terkini untuk mengembangkan bisnis di era digital. SEO, Social
                'price' => 59000,
                'category_id' => $bisnis->id,
            ],
        ],
    ];

    foreach ($ebooks as $data) {
        // Buat file PDF dummy di private storage
        $filename = 'ebooks/' . Str::random(20) . '.pdf';
        \Storage::disk('private')->put($filename, '%PDF-1.4 dummy content');

        Ebook::create([
            'seller_id' => $penjual->id,
            'category_id' => $data['category_id'],
            'title' => $data['title'],
            'slug' => Str::slug($data['title']) . '-' . Str::random(5),
            'description' => $data['description'],
            'price' => $data['price'],
            'cover' => null,
            'file' => $filename,
            'status' => 'aktif',
        ]);
    }
}
```

database/seeder/UserSeeder.php

```
<?php

namespace Database\Seeders;

use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\Hash;
use App\Models\User;

class UserSeeder extends Seeder
{
    public function run(): void
    {
        User::create([
            'name'      => 'Admin',
            'email'     => 'admin@ebook.com',
            'password'  => Hash::make('password'),
            'role'      => 'admin',
        ]);

        User::create([
            'name'      => 'Penjual Demo',
            'email'     => 'penjual@ebook.com',
            'password'  => Hash::make('password'),
            'role'      => 'penjual',
        ]);

        User::create([
            'name'      => 'Pembeli Demo',
            'email'     => 'pembeli@ebook.com',
            'password'  => Hash::make('password'),
            'role'      => 'pembeli',
        ]);
    }
}
```


routes/auth.php

<?php

```
use App\Http\Controllers\Auth\AuthenticatedSessionController;
use App\Http\Controllers\Auth\ConfirmablePasswordController;
use App\Http\Controllers\Auth\EmailVerificationNotificationController;
use App\Http\Controllers\Auth\EmailVerificationPromptController;
use App\Http\Controllers\Auth\NewPasswordController;
use App\Http\Controllers\Auth\PasswordController;
use App\Http\Controllers\Auth\PasswordResetLinkController;
use App\Http\Controllers\Auth\RegisteredUserController;
use App\Http\Controllers\Auth\VerifyEmailController;
use Illuminate\Support\Facades\Route;

Route::middleware('guest')->group(function () {
    Route::get('register', [RegisteredUserController::class, 'create'])
        ->name('register');

    Route::post('register', [RegisteredUserController::class, 'store']);

    Route::get('login', [AuthenticatedSessionController::class, 'create'])
        ->name('login');

    Route::post('login', [AuthenticatedSessionController::class, 'store']);

    Route::get('forgot-password', [PasswordResetLinkController::class, 'create'])
        ->name('password.request');

    Route::post('forgot-password', [PasswordResetLinkController::class, 'store'])
        ->name('password.email');

    Route::get('reset-password/{token}', [NewPasswordController::class, 'create'])
        ->name('password.reset');

    Route::post('reset-password', [NewPasswordController::class, 'store'])
        ->name('password.store');
});

Route::middleware('auth')->group(function () {
    Route::get('verify-email', EmailVerificationPromptController::class)
        ->name('verification.notice');

    Route::get('verify-email/{id}/{hash}', VerifyEmailController::class)
        ->middleware(['signed', 'throttle:6,1'])
        ->name('verification.verify');

    Route::post('email/verification-notification', [EmailVerificationNotificationController::class, 'store'])
        ->middleware('throttle:6,1')
        ->name('verification.send');

    Route::get('confirm-password', [ConfirmablePasswordController::class, 'show'])
        ->name('password.confirm');

    Route::post('confirm-password', [ConfirmablePasswordController::class, 'store']);

    Route::put('password', [PasswordController::class, 'update'])->name('password.update');

    Route::post('logout', [AuthenticatedSessionController::class, 'destroy'])
        ->name('logout');
});
```

routes/console.php

```
<?php
```

```
use Illuminate\Foundation\Inspiring;  
use Illuminate\Support\Facades\Artisan;
```

```
Artisan::command('inspire', function () {  
    $this->comment(Inspiring::quote());  
})->purpose('Display an inspiring quote');
```

[illegible]